

Reach Value Request Web Application — Product Documentation

Author: Manus AI **Date:** July 30, 2026

Executive Summary

This document provides a comprehensive product-level and information-architectural specification for the **Reach Value Request Web Application**. The application is designed to serve Analysts who need domain reach values (primarily from Similarweb API) and Administrators who manage the data pipeline, user access, and the underlying database. The document covers Business Cases, Information Architecture, User Experience, Technical Implementation, Backend Services, DevOps, Edge/Failure Cases, and Deployment & Maintenance — grounded in current industry practices and technical feasibility.

1. Business Cases and Audience Focus

1.1 Analyst (Requester / End User)

The primary audience for this application consists of Analysts who rely on web traffic reach data to perform Article curation, Analysis data collection, Outlet Tiering, and client-facing reporting. Currently, obtaining reach values requires a manual, asynchronous process where analysts must submit requests to administrators and wait for a response. This introduces delays into the analytical workflow, increases the risk of data becoming stale, and creates unnecessary administrative overhead.

The Analyst interface is therefore designed around the principle of **self-service speed**. An analyst should be able to enter a domain name, receive an immediate response if the data already exists in the master database, or submit a single-click request if the data is absent. The system eliminates the need for email chains, spreadsheet exchanges, and manual follow-ups. Features such as bulk domain submission, exportable reach sheets, and multi-reach-provider views further enhance the analyst's analytical toolkit by reducing the time spent on data acquisition and increasing the time available for actual analysis.

1.2 Admin (Management / Request Provider)

Administrators represent the operational backbone of the platform. Their responsibilities span data management (importing and exporting DBO sheets), request fulfillment (triggering Similarweb API fetches for new domains), user lifecycle management (onboarding, role assignment, access revocation), and system oversight (monitoring logs, tracking API credit consumption, Credit Limiter and ensuring data accuracy).

The Admin interface is designed around the principle of **control and visibility**. A single dashboard provides a high-level overview of system health, User Management, Import and Exporting and Log activity. From this dashboard, administrators can access the users list, activity logs, access management, and manage the master database. Because the Similarweb API operates on a data credit model (where each API response consumes a finite number of credits), the admin interface also serves as a cost-control mechanism by centralizing all external API calls behind a single managed workflow.

1.3 Comparative Feature Matrix

Capability	Analyst	Admin
Search for Reach Values	Yes	Yes
Submit New Domain Request	Yes (User can get on their own)	Yes
Export Reach Sheet	Yes	Yes
Import DBO Sheet (Bulk)	No	Yes
Trigger Similarweb API Fetch	Yes	Yes
User Management	No	Yes
View System Logs and Activity	No	Yes
Real-time Notifications	Yes	Yes
Multi Reach Provider Selection	Yes	Yes

2. Information Architecture

2.1 Role-Based Routing and Portal Separation

The application employs a **role-based routing** strategy where, upon successful authentication, users are directed to their respective portals. This separation ensures that each user role is presented with a focused, purpose-built interface that minimizes cognitive load and prevents accidental access to restricted functionality.

Portal	Primary Routes	Purpose
Analyst Portal	<code>/search,/notifications, /profile, /export</code>	Search domains, view reach data, view media type, view date/ month of the value
Admin Portal	<code>/dashboard, /website_traffic, /user_activity, /import-export, /users, /logs, /settings, /notifications</code>	Limit API Credits, import/export data, manage users, monitor system

The authentication layer acts as the single decision point for role determination. Once the JWT (JSON Web Token) is validated and the user's role is extracted, the frontend router conditionally renders the appropriate portal layout and navigation structure. Unauthorized route access results in a redirect to the login page or a "403 Forbidden" page, depending on whether the user is authenticated but lacks the required role.

2.2 Data Architecture

The backend data model is organized around four core entities that interact through well-defined relationships.

Entity	Description	Key Fields
Domain	A record of a website domain and its reach values	<code>id, domain_name, reach_value, provider, last_updated, data_source</code>

Entity	Description	Key Fields
User	An authenticated user with a defined role	id, email, role, status, last_login
ActivityLog	A record of system actions for audit and debugging	id, user_id, resource_type, resource_id, timestamp, details, searched_domain_url, reach_value, search_activity_count

The relationship between these entities is structured as follows. A **User** (Analyst) search for a specific **Domain**. If the domain already exists in the master database, the request is auto-fulfilled and the reach value is returned. If the domain does not exist, then code calls the SimilarWeb API in fetching the domain value, displayed to the user and then stored in the master database.

2.3 Access Management and Collision Avoidance (Requesting from API)

To prevent race conditions and data collisions during concurrent access, the system employs the following strategies. **Optimistic Locking** is used on the Domain entity, where each record carries a version number. If two users attempt to call the same new domain simultaneously, the second write is rejected and the user is prompted to refresh and retry. **Database Transactions** ensure that the sequence of creating a Domain record, updating a ReachRequest status, and logging the activity either completes entirely or rolls back entirely. **Queue-Based Serialization** for Similarweb API calls ensures that only one external API request is processed at a time per data credit bucket, preventing rate limit violations.

3. User Experience and Interaction

3.1 Analyst Portal UX

The Analyst portal is designed with a **single-focus paradigm**. Upon login, the user is presented with a prominent search bar occupying the center of the screen — analogous to how Google presents its homepage. The search bar supports auto-complete suggestions that are fetched from a lightweight endpoint returning only domain names (not heavy reach data), ensuring sub-100ms response times. As the analyst types, a dropdown of matching domains appears, and selecting a result immediately displays the cached reach value, the data source, and the last update timestamp.

If the entered domain does not exist in the database, the system presents a contextual action: a clearly visible "Get Reach" button. Clicking this button creates a ReachRequest record and provides the analyst with a loading screen that says, "Data fetching" when the result is fetched it shows in the same UI as usual output view.

Design Rationale: The single-focus paradigm reduces the number of interactions required to retrieve data. By presenting the search bar as the primary and dominant UI element, the interface communicates its core purpose unambiguously, reducing the learning curve for new users.

3.2 Admin Portal UX

The Admin portal is structured as a **dashboard-first interface**. Upon login, the admin sees a summary view containing key metrics: total New Reach Value Requested, total User Logged in, API credits remaining, and the most recent bulk import timestamp. From this dashboard, the admin can navigate to dedicated modules via a persistent left-hand navigation rail.

3.3 Notification System

Both portals feature an in-app notification center accessible via a bell icon in the header. For Analysts, notifications are triggered when admin side new reach values are updated. For Admins, notifications are triggered when new requests are submitted(no action needed), when bulk imports complete, alerts on too much new reach value requested and when system-level alerts (e.g., low API credits) are raised.

Recommendation: For critical notifications (e.g., a system outage or a sudden spike in requests), consider supplementing in-app notifications with email alerts to ensure Admins are alerted even when they are not actively using the platform.

3.4 UX Improvement Suggestions

Batch Request Submission should be considered as an enhancement. Currently, analysts submit one domain at a time. A batch submission mode, where analysts can paste or upload a list of domains, would significantly improve efficiency for analysts working on large competitive sets. The system would then process these as individual ReachRequest records but present them as a grouped batch to the admin.

Domain Normalization is another critical UX improvement. When an analyst enters a domain, the system should automatically strip common prefixes (e.g., <https://>, [www.](http://)) and trailing slashes to ensure consistent matching against the database. This prevents duplicate requests for the same domain entered in different formats (e.g., example.com vs. www.example.com).

Progressive Disclosure should be applied to the Admin interface. Instead of overwhelming the admin with all available controls upfront, advanced features (e.g., raw database export, API credit configuration) should be accessible through secondary navigation levels. This keeps the primary workflow clean and accessible.

4. Technical Implementation

4.1 Frontend Architecture

The frontend is built as a **Single-Page Application (SPA)** using React.js with TypeScript for type safety. The rationale for this stack is its maturity, extensive ecosystem, and strong support for component-based architecture, which aligns well with the dual-portal design (Analyst and Admin interfaces share common components but differ in layout and available features).

Layer	Technology	Rationale
UI Framework	React 18+ with TypeScript	Component reusability, strong typing, large ecosystem
Routing	React Router v6	Role-based route guards, nested layouts
State Management	Zustand or Redux Toolkit	Lightweight global state for auth, notifications, and request queue
HTTP Client	Axios	Request/response interceptors for JWT injection and error handling
Real-time Updates	Server-Sent Events (EventSource API)	Lightweight unidirectional push for notifications
Styling	Tailwind CSS	Utility-first, rapid development, consistent design system
Auto-complete	Fuse.js (fuzzy search on lightweight domain list)	Fast, client-side filtering with typo tolerance

The SPA architecture ensures that the Analyst and Admin portals are served as a single application bundle, with route-level code splitting to ensure that analysts do not download the admin-specific JavaScript chunks (and vice versa). This optimizes initial load times for both user types.

4.2 Domain Input Handling

The domain input field is the most critical interaction point in the entire application. It must handle a variety of user inputs gracefully. The system normalizes all inputs by stripping protocol prefixes ([http://](#), [https://](#)), subdomain prefixes ([www.](#)), trailing slashes, and query parameters. The normalized domain is then used for both the database lookup and, if necessary, the Similarweb API call. This normalization prevents duplicate records and ensures consistent data retrieval.

4.3 Similarweb API Integration Constraints

The Similarweb API operates under several constraints that directly impact the technical implementation. The API rate limit is set to **10 requests per second** per account [1]. The pricing model uses **data credits**, where each endpoint response consumes a defined number of credits [3]. The Visits endpoint (the primary source for reach values) costs 1 data credit per result [4]. Historical data goes back 37 months, with granularity options of monthly, weekly, or daily. The country parameter is required and defaults to [us](#).

These constraints dictate that all Similarweb API calls must be proxied through the backend. Exposing the API key on the frontend is a security risk and would bypass the queue-based rate limiting that protects the account from exceeding the 10 requests/second limit.

5. Backend Services

5.1 API Layer

The backend exposes a RESTful API that serves as the single interface between the frontend and all data sources. The API is organized into logical resource groups.

Resource Group	Endpoints	Description
Auth	POST /auth/login , POST /auth/register , POST /auth/refresh	User authentication, registration, and token refresh

Resource Group	Endpoints	Description
Domains	GET /domains/search?q=, GET /domains/:id, POST /domains/bulk-import	Domain search, retrieval, and bulk import
Requests (By the user themselves but won't escalate to admin)	GET /requests, POST /requests, PATCH /requests/:id, GET /requests/:id/status	Request creation, listing, status updates
Users	GET /users, POST /users, PATCH /users/:id, DELETE /users/:id	User CRUD operations (Admin only)
Notifications	GET /notifications, PATCH /notifications/:id/read	Notification listing and read status
Logs	GET /logs, GET /logs/:id	Activity log retrieval (Admin only)
Export	GET /export/domains.csv, GET /export/domains.xlsx	Data export in CSV and Excel formats (Admin only)

5.2 Authentication and Authorization

Authentication is implemented using **JWT (JSON Web Tokens)** with a short-lived access token (15 minutes) and a long-lived refresh token (7 days). The refresh token is stored in an HTTP-only, secure cookie to prevent XSS (Cross-Site Scripting) attacks. The access token is passed in the **Authorization** header for subsequent API requests.

Authorization is enforced via **RBAC middleware** that intercepts every protected route. The middleware decodes the JWT, extracts the user's role, and compares it against the required permissions for the requested endpoint. If the user lacks the required role, a **403 Forbidden** response is returned.

5.3 Request Processing Service (Its not necessary, user can fetch by themselves)

The core of the backend is the **Request Processing Service**, which operates as an asynchronous background worker. When an admin triggers the fulfillment of a pending reach request, the following sequence occurs. The request is placed into a **Redis Queue** (using BullMQ or a similar library). The worker picks up the request, normalizes the domain name, and calls the Similarweb API's Visits endpoint. If the API returns successfully, the reach data is written to the master database, the request status is updated to "Completed," and a notification is pushed to the requesting analyst via SSE. If the API returns an error (e.g., 429 rate limit, 404 domain not found), the request is either retried (with exponential backoff) or marked as "Failed" with an appropriate error message.

Status	Meaning	Next Action
Pending	Request submitted, awaiting admin action	Admin selects and fulfills
Processing	API fetch in progress	Wait for worker completion
Completed	Data retrieved and stored	Notify analyst
Failed	API error or invalid domain	Admin can retry or reject
Cancelled	Request withdrawn by analyst	No further action

5.4 Database Design

PostgreSQL is the recommended database engine due to its strong support for structured data, transactional integrity, and JSONB columns for flexible metadata storage.

-- Core Schema

CREATE TABLE users (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

email VARCHAR(255) UNIQUE NOT NULL,

password_hash VARCHAR(255) NOT NULL,

role VARCHAR(20) NOT NULL CHECK (role IN ('analyst', 'admin')),

```
status VARCHAR(20) DEFAULT 'active' CHECK (status IN ('active', 'inactive', 'suspended')),  
  
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  
last_login TIMESTAMP WITH TIME ZONE  
);
```

```
CREATE TABLE domains (  
  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
domain_name VARCHAR(255) UNIQUE NOT NULL,  
  
reach_value BIGINT,  
  
provider VARCHAR(50) NOT NULL DEFAULT 'similarweb',  
  
country VARCHAR(10) NOT NULL DEFAULT 'world',  
  
granularity VARCHAR(20) DEFAULT 'monthly',  
  
data_source VARCHAR(50) CHECK (data_source IN ('api_fetch', 'bulk_import', 'manual')),  
  
version INTEGER DEFAULT 1,  
  
last_updated TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

```
CREATE TABLE reach_requests (  
  
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  
domain_name VARCHAR(255) NOT NULL,  
  
requested_by UUID NOT NULL REFERENCES users(id),  
  
assigned_to UUID REFERENCES users(id),
```

status VARCHAR(20) NOT NULL DEFAULT 'pending'

CHECK (status IN ('pending', 'processing', 'completed', 'failed', 'cancelled')),

priority INTEGER DEFAULT 0,

notes TEXT,

created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

fulfilled_at TIMESTAMP WITH TIME ZONE

);

CREATE TABLE activity_logs (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

user_id UUID REFERENCES users(id),

action_type VARCHAR(50) NOT NULL,

resource_type VARCHAR(50) NOT NULL,

resource_id UUID,

details JSONB,

timestamp TIMESTAMP WITH TIME ZONE DEFAULT NOW()

);

CREATE TABLE notifications (

id UUID PRIMARY KEY DEFAULT gen_random_uuid(),

user_id UUID NOT NULL REFERENCES users(id),

type VARCHAR(50) NOT NULL,

title VARCHAR(255) NOT NULL,

message TEXT NOT NULL,

```
is_read BOOLEAN DEFAULT FALSE,  
  
created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
  
);
```

5.5 Notification Delivery

The notification system uses **Server-Sent Events (SSE)** for real-time, unidirectional push from the server to the client. SSE is preferred over WebSockets for this use case because the communication pattern is inherently one-directional (server pushes notifications to the client, the client does not send data back over the same channel) and SSE operates over standard HTTP, making it easier to debug, proxy, and scale through load balancers without requiring special WebSocket-aware infrastructure [2].

On the server side, each authenticated user maintains an SSE connection identified by their user ID. When a notification is created (e.g., a request is fulfilled), the server pushes an event to all connected clients matching that user ID. The frontend uses the native **EventSource** API to receive and display these events.

5.6 Import/Export Service

The bulk import service accepts CSV or Excel files uploaded by the admin. The file is parsed server-side, each row is validated against the expected schema (domain name, reach value, provider, date), and valid rows are inserted into the master database using a batch insert operation for performance. Invalid rows are collected and returned as an error report, allowing the admin to correct and re-upload. The export service generates CSV or Excel files from the master database, optionally filtered by date range or provider.

6. DevOps

6.1 Infrastructure and Deployment Architecture

The application is containerized using **Docker** with a multi-container setup defined via Docker Compose for development and a container orchestration platform (Kubernetes or AWS ECS) for production.

Component	Container	Technology
Frontend	web-frontend	Nginx serving React build artifacts
Backend API	web-backend	Node.js (Express/Fastify) or Python (FastAPI)
Background Worker	web-worker	Celery (Python) or BullMQ (Node.js)
Database	db	PostgreSQL 15+
Cache / Queue	redis	Redis 7+ (for session caching and task queue)

6.2 CI/CD Pipeline

A continuous integration and continuous deployment (CI/CD) pipeline is configured using GitHub Actions or GitLab CI. The pipeline executes on every push to the `main` branch and includes the following stages.

Stage 1: Lint and Type Check. TypeScript compiler and ESLint run to ensure code quality and catch type errors.

Stage 2: Unit and Integration Tests. Jest or Pytest runs the full test suite. Coverage thresholds (minimum 80%) are enforced.

Stage 3: Build and Push. Docker images are built and pushed to a container registry (Docker Hub, AWS ECR, or GitHub Container Registry).

Stage 4: Deploy to Staging. The staging environment is updated. Smoke tests verify that the application is functional.

Stage 5: Manual Approval and Production Deploy. After manual approval, the production environment is updated using a Blue-Green deployment strategy to minimize downtime.

6.3 Monitoring and Observability

Tool Category	Tool	Purpose
APM	Datadog / New Relic / Grafana	Monitor API latency, error rates, throughput

Tool Category	Tool	Purpose
Logging	ELK Stack / AWS CloudWatch	Centralized log aggregation and search
Uptime	Pingdom / Better Stack	External uptime monitoring with alerting
Error Tracking	Sentry	Capture and group frontend/backend errors
API Monitoring	Custom dashboard	Track Similarweb API credit consumption and rate limit status

7. Test, Edge, Failure, and Fallback Cases

7.1 Testing Strategy

Test Type	Scope	Tools
Unit Testing	Individual functions and components	Jest / Pytest
Integration Testing	API endpoints, database interactions, queue processing	Supertest / HTTPX
End-to-End Testing	Full user workflows (login, search, request, fulfill)	Cypress / Playwright
Load Testing	Concurrent search requests, bulk import performance	k6 / Locust
Security Testing	Authentication bypass, SQL injection, XSS, CSRF	OWASP ZAP / Snyk

7.2 Edge Cases

The following edge cases have been identified and addressed in the design.

Duplicate Domain Requests. If an analyst submits a request for a domain that is already in the database, the system should detect the duplicate and return the existing value rather than creating a new one. This prevents redundant API calls and data credit waste.

Domain Normalization Edge Cases. Domains entered as <http://example.com/>, <https://www.example.com>, <example.com/path>, and <EXAMPLE.COM> should all resolve to the same database record. The normalization logic must handle all these variations consistently.

Bulk Import with Mixed Validity. If an import file contains 950 valid rows and 50 invalid rows, the system should process the valid rows and return a detailed error report for the 50 invalid rows, rather than rejecting the entire batch.

7.3 Failure Cases

Failure Scenario	Detection	System Response	User Experience
Similarweb API returns 429 (Rate Limit)	HTTP status code check	Exponential backoff, retry after 5/10/30/60 seconds	Request stays in "Processing" state, analyst sees "Queued" status
Similarweb API returns 404 (Domain Not Found)	HTTP status code check	Request marked as "Failed" with reason	Admin sees failure reason, can retry with corrected domain or notify analyst
Similarweb API returns 500 (Server Error)	HTTP status code check	Retry after delay, up to 3 attempts	Request remains "Processing" until retries exhausted
Database connection lost	Connection pool health check	Display maintenance message, queue incoming writes	Users see "System temporarily unavailable" banner
Redis queue unavailable	Queue health check	Fallback to synchronous processing or queue in memory	Admin sees alert; requests process sequentially
SSE connection drops	Client-side heartbeat check	Auto-reconnect with exponential backoff	Notifications resume automatically; missed events fetched on reconnect

7.4 Fallback Mechanisms

The primary fallback mechanism is the **database export capability**. If the application becomes unavailable, admins can export the master reach database to a CSV or Excel file from within the application (before an outage) or from a direct database backup. This exported file can be shared with analysts as a temporary workaround until the system is restored.

For extended Similarweb API outages, the system can display a **service banner** informing users that external data fetches are temporarily unavailable. Analysts can still search the existing cached database, and their requests will be queued for automatic processing once the API recovers.

8. Deployment and Maintenance

8.1 Deployment Strategy

The application follows a **Blue-Green deployment** model. Two identical production environments (Blue and Green) run in parallel. When a new version is ready, it is deployed to the inactive environment. After automated smoke tests pass and manual verification is complete, the load balancer switches traffic from the active to the inactive environment. This ensures zero-downtime deployments and provides an instant rollback mechanism by switching traffic back to the original environment.

Environment	Purpose	Data
Development	Active development and feature testing	Synthetic/mock data
Staging	Pre-production validation, QA testing	Anonymized production-like data
Production	Live application serving real users	Full production data

8.2 Maintenance Procedures

Scheduled Data Refresh. High-priority domains can be configured for periodic data refresh (e.g., weekly or monthly) via a cron job that triggers background workers to re-fetch reach values from the Similarweb API. This ensures that the master database does not become stale for domains that analysts access frequently.

API Credit Monitoring. An automated alert system monitors Similarweb data credit consumption. When usage reaches 75% of the monthly allocation, a warning notification is sent to the admin. At 90%, a critical alert is triggered. At 100%, the system enters a "credit exhaustion" state where new API fetches are blocked until credits are replenished or the billing period resets.

Database Maintenance. Regular vacuum and analyze operations on the PostgreSQL database ensure optimal query performance. Old activity logs (beyond a defined retention period, e.g., 90 days) can be archived to cold storage to reduce database size and improve backup times.

Security Audits. Quarterly security audits should be conducted, including review of access logs for anomalous activity, validation of RBAC configurations, and penetration testing of the authentication flow.

Backup Strategy. Automated daily backups of the PostgreSQL database should be stored in a separate, encrypted location (e.g., AWS S3 with server-side encryption). Point-in-time recovery should be enabled to allow restoration to any moment within the retention window.

Questions and Gaps Identified

The following questions and observations arise from the analysis of the concept. Addressing these will strengthen the final implementation.

1. Multi-Provider Reach. The concept mentions "Multi Reach Provider" as an Analyst feature. If this implies integrating additional data providers beyond Similarweb (e.g., Semrush, Ahrefs, SimilarSites), the backend architecture must be extended to support a provider abstraction layer. Each provider would be represented as a pluggable service with its own authentication, rate limiting, and data mapping logic. If it refers only to Similarweb's multiple data endpoints (Visits, Page Views, Bounce Rate), then no additional provider integration is needed.

2. Admin Notification Channel. The concept specifies in-app notifications for both roles. For Admins, who may not be actively monitoring the platform at all times, consider whether email notifications should supplement in-app alerts for high-priority events (e.g., a sudden influx of requests or a system error). This ensures that critical requests are not left unattended.

3. Data Retention Policy. The concept does not specify how long reach values should be retained in the database. Similarweb's API provides up to 37 months of historical data. A retention policy should be defined — for example, retaining the latest monthly value per domain indefinitely, while archiving or purging older granular data after a defined period.

4. Analyst Request Limits. There is no mention of limits on the number of requests an analyst can submit within a given period. Without rate limiting at the user level, a single analyst could flood the request queue, consuming admin time and API credits. Consider implementing per-user daily or weekly request quotas.

5. DBO Sheet Format Specification. The concept references a "DBO Reach sheet" for import and export, but does not specify the expected column structure, file format, or naming conventions. A formal specification should be created to ensure consistency across imports.

References

[1]: Similarweb API Documentation. "Rate Limit."

<https://developers.similarweb.com/docs/rate-limit>

[2]: Ably. "WebSockets vs Server-Sent Events: Key differences and which to use."

<https://ably.com/blog/websockets-vs-sse>

[3]: Similarweb Documentation. "API Pricing & Data Credits."

<https://docs.similarweb.com/api-v5/guides/data-credits-calculations>

[4]: Similarweb API Documentation. "Visits - Total (Desktop & Mobile)."

<https://developers.similarweb.com/reference/visits>